

Decision Trees

Decision Trees are a type of supervised machine learning in which data is continuously split according to a certain parameter and they learn a hierarchy of if/else questions, leading to a decision. They can be used for both classification and regression.

Generally, a decision tree is a flowchart-like tree structure, where

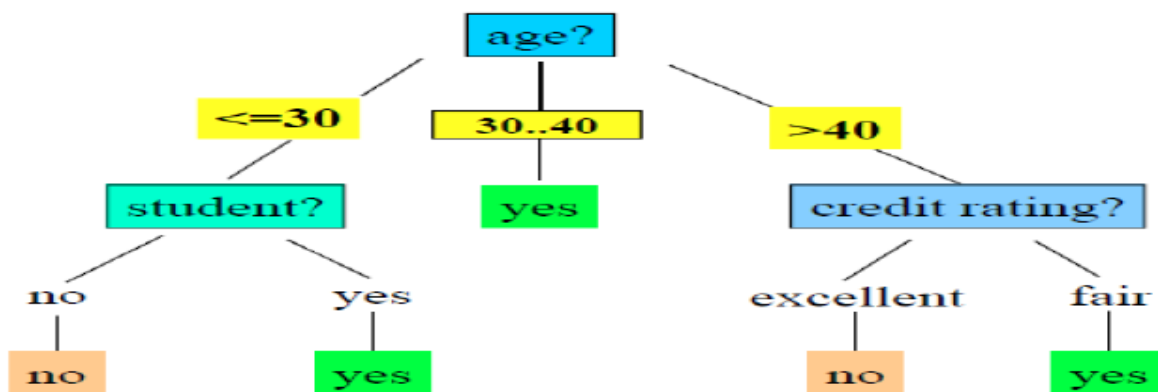
- Each internal node denotes a test on an attribute (decision nodes, where data is split).
- Each branch represents an outcome of the test.
- Each leaf node holds a class label (used to denote decision or final outcome).
- The topmost node in a tree is the root node.

Example

Consider the following training dataset:

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31...40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31...40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31...40	medium	no	excellent	
31...40	high	yes	fair	
>40	medium	no	excellent	

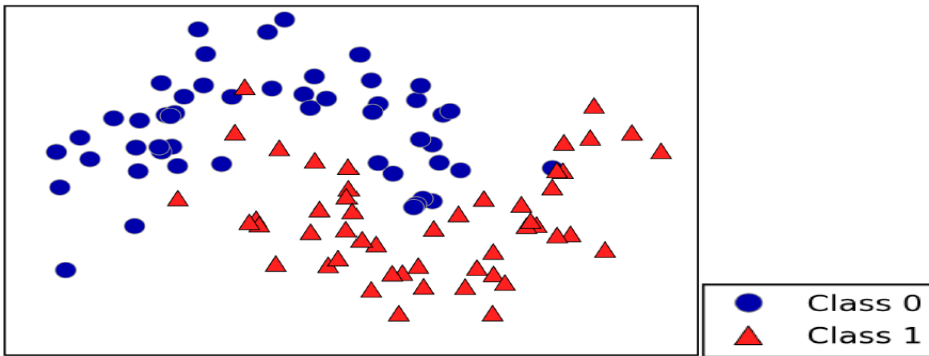
Output: A Decision Tree for “*buys_computer*”



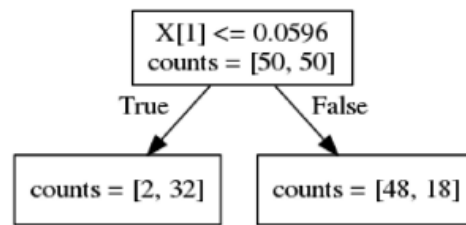
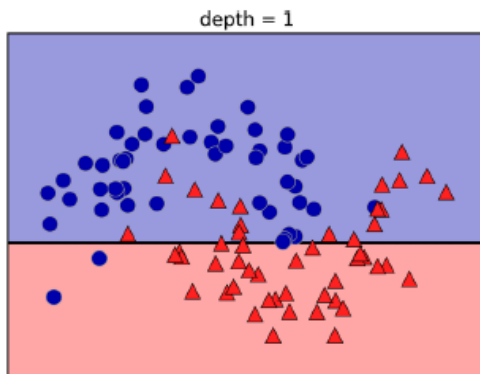
The decision nodes are questions like ‘What’s the age?’, ‘Is he a student?’, and ‘What’s is his credit ratings?’ And the leaves, which are outcomes like either ‘yes’, or ‘no’.

Building decision trees

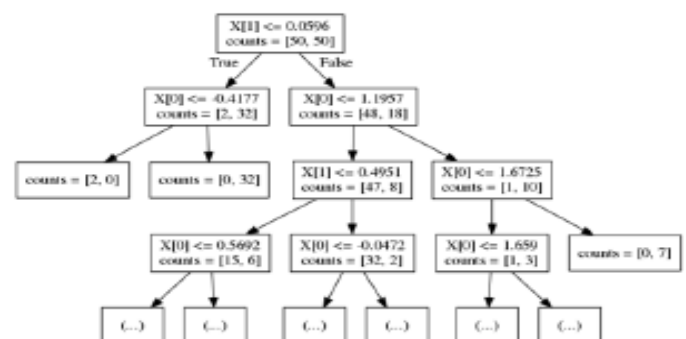
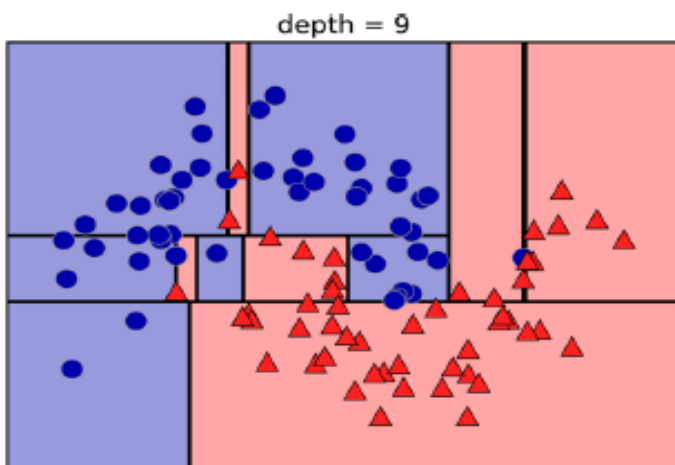
Learning a decision tree is learning the sequence of if/else questions. In the machine learning setting, these questions are called *tests* (not to be confused with the test set). The data is usually represented as continuous features such as in the 2D dataset shown in the figure below. The tests that are used on continuous data are of the form “Is feature *i* larger than value *a*?”



To build a tree, the algorithm searches over all possible tests and finds the one that is most informative about the target variable, in our case at $x[1]=0.0596$. Splitting the dataset vertically yields the most information; it best separates the points in class one from the points in class two. The root node, represents the whole dataset. The split is done by testing whether $x[1] \leq 0.0596$, indicated by a black line. If the test is true, a point is assigned to the left node. These two nodes correspond to the top and bottom regions shown in the diagram. Even though the first split did a good job of separating the two classes, the bottom region still contains points belonging to class 0, and the top region still contains points belonging to class 1.



The recursive partitioning of the data is repeated until each region in the partition only contains a single target value (a single class or a single regression value). A leaf of the tree that contains data points that all share the same target value is called *pure*. The final partitioning for this dataset is shown below



A prediction on a new data point is made by checking which region of the partition of the feature space the point lies in, and then predicting the majority target (or the single target in the case of pure leaves) in that region. The region can be found by traversing the tree from the root and going left or right, depending on whether the test is fulfilled or not.

Decision trees in scikit-learn

Decision trees in scikit-learn are implemented in the *DecisionTreeClassifier* and *DecisionTreeRegressor* classes.

Example

In the following example, you are required to implement *DecisionTreeClassifier* on Iris dataset, which is one of datasets that is included in scikit-learn package and has four features (sepal length, sepal width, petal length and petal width) and each instance has a class of either setosa (class 0), versicolor (class 1), or virginica (class 2) as shown below.

sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
7.2	3.2	6.0	1.8	virginica
5.1	3.5	1.4	0.2	setosa
6.0	2.2	5.0	1.5	virginica
5.4	3.0	4.5	1.5	versicolor
4.4	3.0	1.3	0.2	setosa
6.9	3.2	5.7	2.3	virginica
5.6	3.0	4.1	1.3	versicolor

```
#import the necessary libraries
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris

#load the iris dataset
iris=load_iris()

#split the dataset
X_train, X_test, y_train, y_test=train_test_split(iris.data, iris.target,
random_state=0)

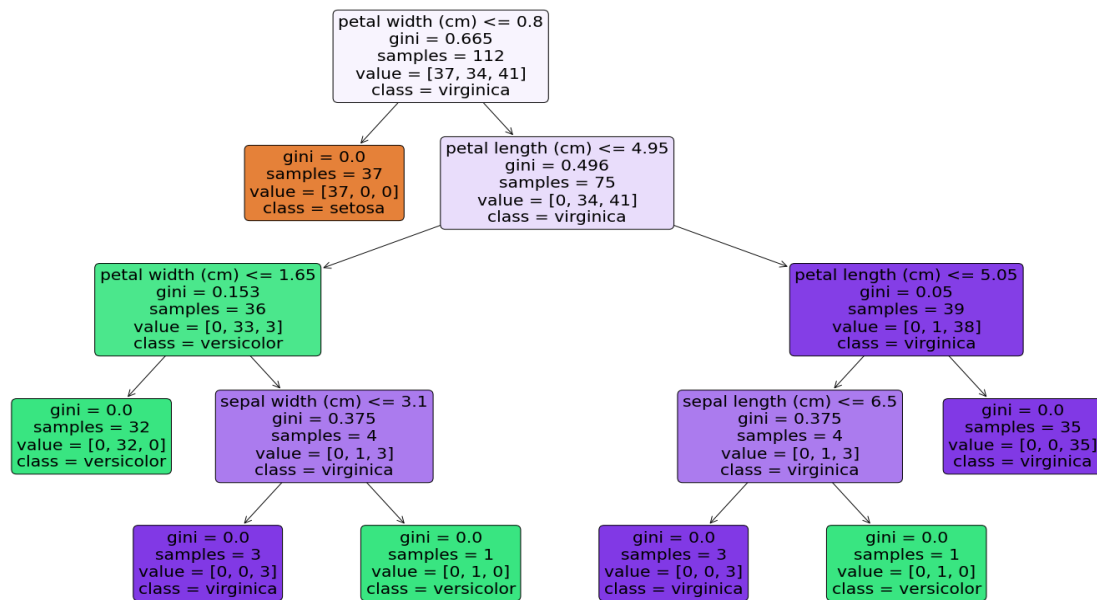
#build the model
tree_model=DecisionTreeClassifier()
tree_model.fit(X_train,y_train)

#visualize the tree
from sklearn import tree
fig=plt.figure(figsize=(25,15))
tree.plot_tree(tree_model,filled=True,rounded=True,
                feature_names=iris.feature_names,
                class_names=["setosa", "versicolor","virginica"])
```

When visualising the tree, you can set an option to color the nodes to reflect the majority class in each node and pass the class and features names so the tree can be properly labeled.

The visualization of the tree provides a great in-depth view of how the algorithm makes predictions, and is a good example of a machine learning algorithm that is easily explained to non-experts

Output



The samples gives the number of samples in that node, while value provides the number of samples per class.

Alternatively, trees can be visualized by using the `export_graphviz` function from the `tree` module. This writes a file in the `.dot` file format, which is a text file format for storing graphs

```
from sklearn.tree import export_graphviz
export_graphviz(tree_model, out_file="tree_model.dot",
class_names=["setosa", "versicolor", "virginica"],
feature_names=iris.feature_names, impurity=False, filled=True)
```

You can read this file and visualize it, using the `graphviz` module (or you can use any program that can read `.dot` files). You might need to install `graphviz`

```
import graphviz
with open("tree_model.dot") as f:
    dot_graph = f.read()
graphviz.Source(dot_graph)
```

The tree can also be visualized as follows:

```
tree_rep=tree.export_text(tree_model)
print (tree_rep)
```

Gini Impurity

There are different methods used to split the features. The Gini Impurity, is the simplest hence common. Other methods are Entropy / Information gain albeit more complex

Gini Impurity is a measurement used to build Decision Trees to determine how the features of a dataset should split nodes to form the tree i.e. it determines the likelihood that a data point with the feature will be misclassified if assigned to a class. Thus measures the impurity level of a split.

It yields a number between 0 and 0.5. A small Gini value signifies less impurity hence a suitable split. It is calculated as shown where:

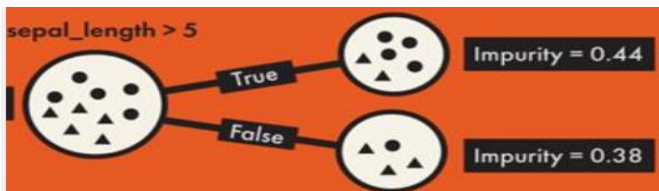
$$Gini(D) = 1 - \sum_{i=1}^k p_i^2$$

Where,

- D is the dataset
- k is the number of classes.
- P is the probability of samples belonging to class i

Weighted Gini Impurity

Assume that algorithm picks the feature sepal length for a node, it creates a threshold (e.g. 5cm) and then splits the data point into those whose attribute value is above the threshold and those that are below or equal to it. If a pattern exists then one group will tend to have data points that belong to one class and the other group will have a different class. There will be some impurities e.g., some circle species where triangle are the majority.



We calculate this impurity for each group as follows:

- Gini impurity True (Sepal Length>5)
 - Probability of class circle = 4/6 = 0.67
 - Probability of class triangle = 2/6 = 0.33
 - Gini impurity for True = 1-[(0.67*0.67)+(0.33*0.33)]=0.44
- Similarly Gini impurity False = 0.38

The weighted Gini for all groups in a feature (attribute) is computed as follows

$$Gini_A(D) = \frac{n_1}{n} Gini(D_1) + \frac{n_2}{n} Gini(D_2)$$

$$Gini(sepal_length>5)=6/10*0.44 + 4/10*0.38=0.42$$

The process is repeated for other features e.g. petal length > 5 and the feature with the lowest Weighted Gini Impurity is selected for further splitting. A node is pure when the Weighted Gini Impurity is 0. This is when splitting stops. An alternative approach for finding the best split uses the highest information gain (entropy).

Example

In the following example, you are required to implement *DecisionTreeClassifier* estimator on Wisconsin Breast Cancer dataset, which records clinical measurements of breast cancer tumors. Each tumor is labeled as “benign” (for harmless tumors) or “malignant” (for cancerous tumors).

a) Import the required dataset

```
In[4]: from sklearn.datasets import load_breast_cancer
cancer = load_breast_cancer()
print("Shape of cancer data: {}".format(cancer.data.shape))
```

```
Out[3]: Shape of cancer data: (569, 30)
```

b) Importing the required libraries for the decision tree analysis

```
In[5]: from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
```

c) Display the dataset. You can use pandas dataframe

```
import pandas as pd
cancer_df=pd.DataFrame(data=np.c_[cancer.data,cancer.target],
                        columns=np.append(cancer.feature_names,['target']))
cancer_df.sample(10)
```

worst perimeter	worst area	worst smoothness	worst compactness	worst concavity	worst concave points	worst symmetry	worst fractal dimension	target
174.90	2232.0	0.14380	0.38460	0.68100	0.22470	0.3643	0.09223	0.0
88.70	574.4	0.13840	0.12120	0.10200	0.05602	0.2688	0.06888	1.0
116.60	993.6	0.14010	0.15460	0.26440	0.11600	0.2884	0.07371	0.0
95.48	698.7	0.09023	0.05836	0.01379	0.02210	0.2267	0.06192	1.0
108.40	808.9	0.13060	0.19760	0.33490	0.12250	0.3020	0.06846	0.0
108.10	830.5	0.10890	0.26490	0.37790	0.09594	0.2471	0.07463	1.0
68.81	366.1	0.13160	0.09473	0.02049	0.02381	0.1934	0.08988	1.0
184.60	1821.0	0.16500	0.86810	0.93870	0.26500	0.4087	0.12400	0.0
102.20	773.5	0.10180	0.15500	0.12200	0.07071	0.2505	0.06827	1.0

d) Dividing the data into training and testing sets in the ratio of 70:30.

```
In[6]: X_train, X_test, y_train, y_test = train_test_split(cancer.data,
cancer.target, test_size=0.3, random_state=1)
```

e) Building the Decision Tree Model using scikit-learn and determine the accuracy

```
In[7]: tree_model= DecisionTreeClassifier(criterion='entropy')
tree_model.fit(X_train, y_train)
print("Accuracy on training set: {:.3f}"
      .format(tree_model.score(X_train, y_train)))
print("Accuracy on test set: {:.3f}".format(tree_model.score(X_test,
y_test)))
```

```
Out[7]:
Accuracy on training set: 1.000
Accuracy on test set: 0.937
```

Controlling complexity of decision trees

The training set is 100%. This is because the leaves are pure and the tree was grown deep enough that it could perfectly memorize all the labels on the training data i.e. if you don't restrict the depth of a decision tree, the tree can become arbitrarily deep and complex. Unpruned trees are prone to overfitting and not generalizing well to new data.

There are two common strategies to prevent overfitting:

- Stopping the creation of the tree early (also called *pre-pruning*)
- Building the tree but then removing or collapsing nodes that contain little information (also called *post-pruning* or just *pruning*).

Possible criteria for pre-pruning include limiting the maximum depth of the tree (`max_depth`), limiting the maximum number of leaves (`min_samples_leaf`), or requiring a minimum number of points in a node (`min_leaf_nodes`) to keep splitting it.

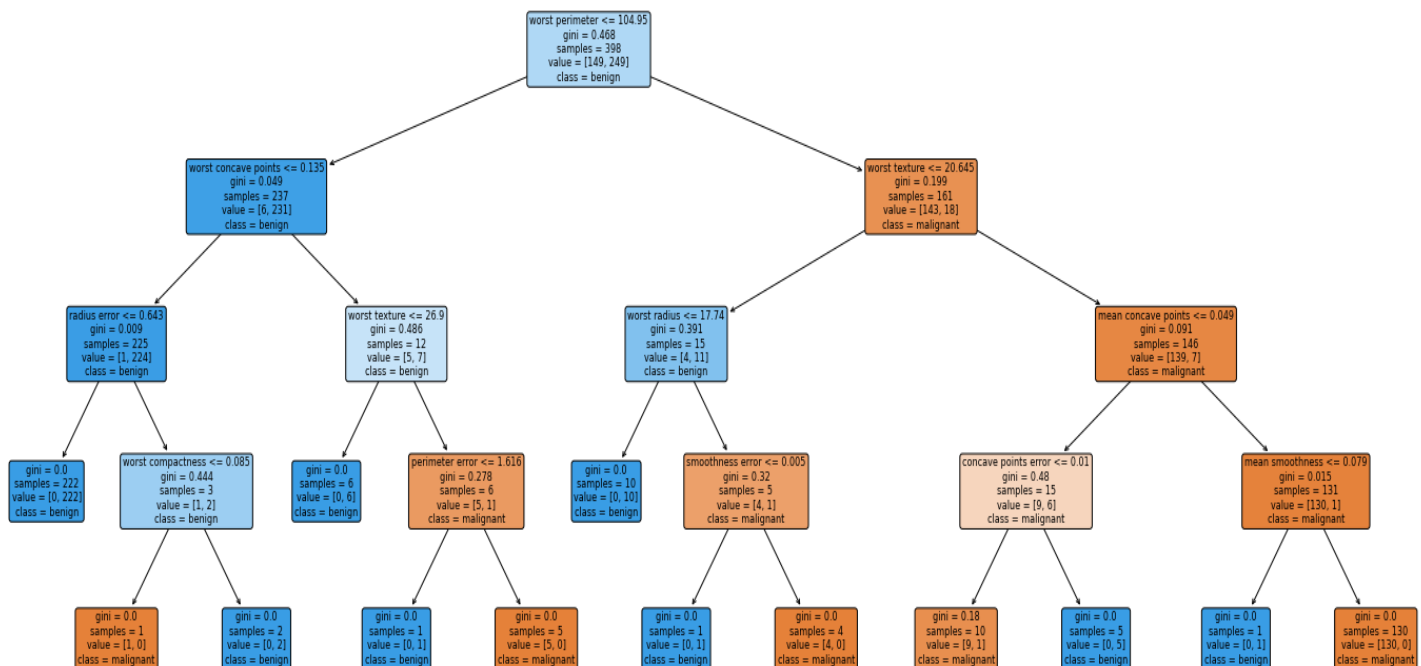
Pre-pruning

scikit-learn only implements pre-pruning, not post-pruning. Apply pre-pruning to the tree, will stop developing the tree before it is perfectly fit to the training data. One option is to stop building the tree after a certain depth has been reached. You can set `max_depth=4`, meaning only four consecutive questions can be asked and this decreases overfitting. This leads to a lower accuracy on the training set (leaves are not as pure), but an improvement on the test set hence generalizing better (decreases overfitting) as shown:

```
In[8]: tree_model = DecisionTreeClassifier(max_depth=4, random_state=0)
Tree_model.fit(X_train, y_train)
print("Accuracy on training set: {:.3f}".
format(tree_model.score(X_train, y_train)))
print("Accuracy on test set: {:.3f}".format(tree_model.score(X_test,
y_test)))
```

```
Out[8]:
Accuracy on training set: 0.988
Accuracy on test set: 0.951
```

```
#Visualise the prepruned tree
import matplotlib.pyplot as plt
from sklearn import tree
fig=plt.figure(figsize=(25,10))
tree.plot_tree(tree_model,filled=True,rounded=True,
feature_names=cancer.feature_names,
class_names=["malignant", "benign"])
```



Feature importance in trees

However, even with a tree of depth four, the tree can still be huge. The leaves with more samples are the useful ones when making predictions. If we visualize with color, we notice the orange nodes (malignant samples) are mostly on one leaf, with the other leaves containing few samples. Thus, instead of looking at the whole tree, we can focus on the ones with more samples. The number of samples is called *feature importance*.

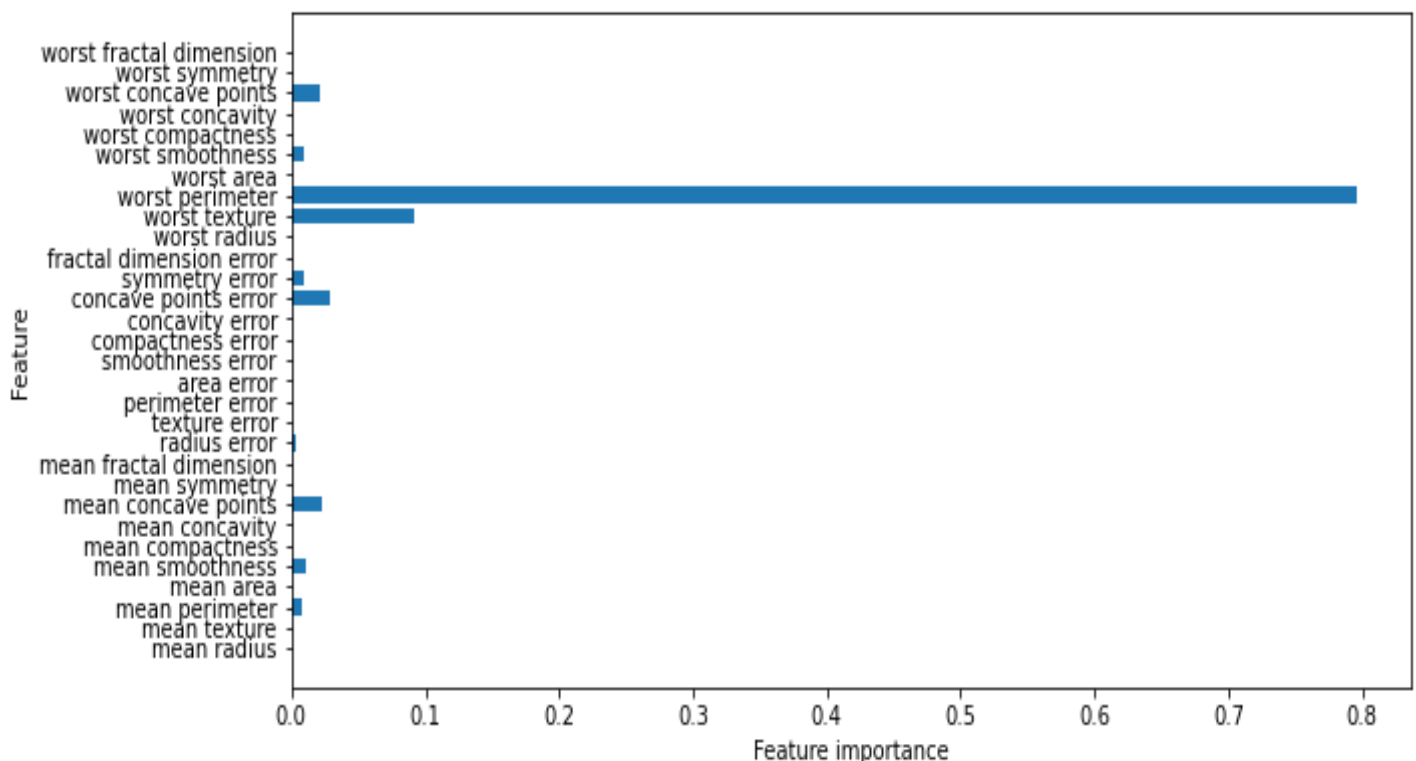
Feature importance provides a number between 0 and 1 for each feature, where 0 means “not used at all” and 1 means “perfectly predicts the target.” The feature importance always sums to 1:

```
In[11]:
print("Feature importances:\n{}".format(tree_model.feature_importances_))
Out[11]:
Feature importances:
[0.          0.          0.00722137  0.          0.01074937  0.
 0.          0.0224442  0.          0.          0.00356254  0.
 0.          0.          0.          0.          0.          0.02924655
 0.00902671  0.          0.          0.0917821  0.79633197  0.
 0.00866564  0.          0.          0.02096955  0.          0.          ]
```

Feature importance's can be visualized using the following:

```
In[12]: import numpy as np
fig=plt.figure(figsize=(10,5))
n_features = cancer.data.shape[1]
plt.barh(range(n_features), tree_model.feature_importances_,
align='center')
plt.yticks(np.arange(n_features), cancer.feature_names)
plt.xlabel("Feature importance")
plt.ylabel("Feature")
```

Out[12]:



The feature used in the top split (“worst perimeter”) is by far the most important feature. This confirms the observation in analyzing the tree that the first level already separates the two classes fairly well.

However, if a feature has a low `feature_importance`, it doesn't mean that this feature is uninformative. It only means that the feature was not picked by the tree, likely because another feature encodes the same information. Notice that linear model coefficients are better because in addition to telling us which features are important, they also tell us which class the features are important for. With feature importance we can tell "worst perimeter" is important, but not whether a high worst perimeter is indicative of a sample being benign or malignant.

Exercise

Use the Pima Indian Diabetes dataset <https://www.kaggle.com/uciml/pima-indians-diabetes-database>, to perform the following tasks:

- a) Import the required libraries for the decision tree analysis and load in the required data
- b) Determine the target and feature variables
- c) Divide the data into training and testing sets in the ratio of 70:30.
- d) Building the Decision Tree Model using scikit-learn
- e) Determine how accurately the classifier predicts the outcome. The accuracy is computed by comparing actual test set values and predicted values.
- f) Visualise the tree model

Regression Trees

Decision tree for regression, is implemented using *DecisionTreeRegressor* and the usage and analysis of regression trees is similar to that of classification trees although the tree-based regression models are not able to *extrapolate*, or make predictions outside of the range of the training data.

Decision tree classifiers have two advantages over many of the algorithms:

- (i) The resulting model can easily be visualized and understood by non-experts (especially for smaller trees)
- (ii) The algorithms are completely invariant to scaling of the data. As each feature is processed separately, and the possible splits of the data don't depend on scaling, no preprocessing like normalization or standardization of features is needed for decision tree algorithms.

The main downside of decision trees classifiers is that even with the use of pre-pruning, they tend to over fit as you don't always know how much pruning to do to avoid over fitting. This can be addressed by combining different types of trees.

Ensembles Methods

Ensembles are methods that combine multiple machine learning models to create more powerful models. Here we look at ensemble trees. Two types that perform well are:

- i) Random forests
- ii) Gradient boosted regression trees

Random Forests

Random forests use a collection of slightly different trees. Basically, if you build many accurate trees, which overfit in different ways, you can reduce the amount of overfitting by averaging their results. To implement this strategy, you need to build different trees. Random forests inject in their trees by selecting the data points used to build a tree and by selecting the features used for splitting randomly.

To build a random forest model, specify the number of trees to build using the *n_estimators* parameter. The larger the number of trees the higher the accuracy and generalizability results.

The algorithm works as follows:

- i) Selects a sample of the data (called a *bootstrap sample*) of the data and then repeats this sampling process it creates a dataset that is as big as the original dataset, (some data points from the original dataset will be missing while some will be repeated).
- ii) Next, a decision tree is built based on this newly created dataset. However, only a subset of the features are used when deciding how to split a node. The number of features selected is controlled by the *max_features* parameter. Each node uses a different subset of the features.

Notice that, if you set `max_features` to all `n` features, no randomness will be injected in the feature selection making all trees similar (increases overfitting), and if we set `max_features` to 1, the splits have no choice on which feature thus can only search over different thresholds for the feature that was selected randomly. Thus, this value should be controlled. (A good rule of thumb selects `max_features=sqrt(n_features)` for classification and `max_features=log2(n_features)` for regression.)

To make a prediction the algorithm first makes a prediction for every tree in the forest. For classification, a “soft voting” strategy is used. Where the class with the highest probability is predicted while for regression, the results are averaged to get the final prediction.

Parameter Selection

As shown below, the random forest gives us an accuracy of 96%, better than the linear models or a single decision tree, even without tuning any parameters. We could adjust the `max_features` setting, to reduce overfitting, however, often the default parameters of the random forest already work quite well.

```
# Random Forest on Breast Cancer Dataset
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier

cancer = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    cancer.data, cancer.target, stratify=cancer.target, random_state=42)
model = RandomForestClassifier(n_estimators=100, random_state=0)
model.fit(X_train, y_train)

print("Accuracy on training set", model.score(X_train, y_train))
print("Accuracy on testing set", model.score(X_test, y_test))
```

Accuracy on training set 1.0
Accuracy on testing set 0.958041958041958

```
1 # Random Forest on Breast Cancer Dataset
2 from sklearn.model_selection import train_test_split
3 from sklearn.datasets import load_breast_cancer
4 from sklearn.ensemble import RandomForestClassifier
5
6 cancer = load_breast_cancer()
7 X_train, X_test, y_train, y_test = train_test_split(
8     cancer.data, cancer.target, stratify=cancer.target, random_state=42)
9 model = RandomForestClassifier(max_features=10, max_depth=4, n_estimators=100, random_state=0)
10 model.fit(X_train, y_train)
11
12 print("Accuracy on training set", model.score(X_train, y_train))
13 print("Accuracy on testing set", model.score(X_test, y_test))
14
```

Accuracy on training set 0.9882629107981221
Accuracy on testing set 0.958041958041958

Feature Importance

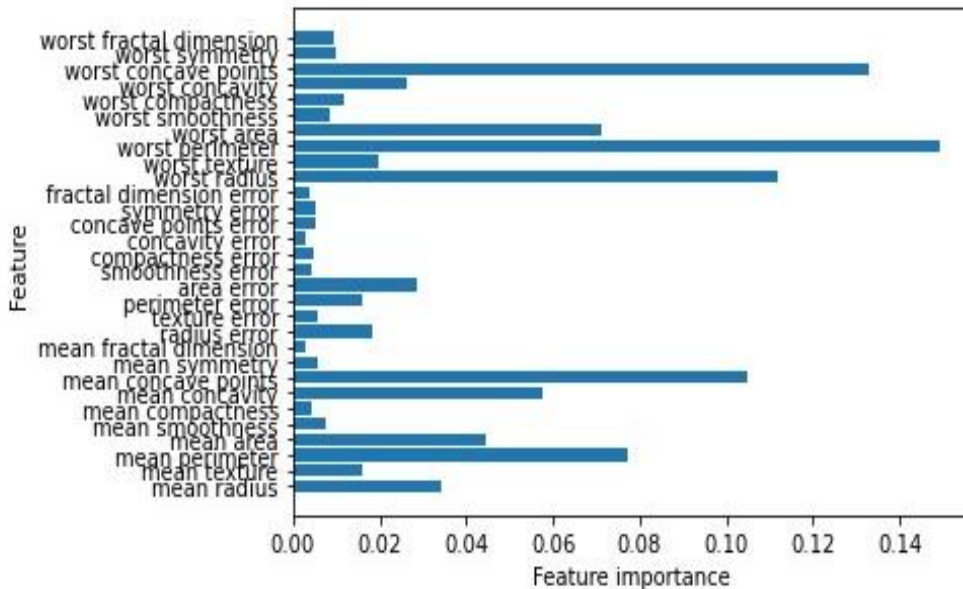
Similar to the decision tree, the random forest provides feature importance's, which are computed by aggregating the importance's of all trees and these are typically more reliable than the ones provided by a single tree. i.e., random forest considers more features as important. The levels of importance for different features might also differ from those of a single tree, thus it captures a much broader picture of the data than a single tree hence improved accuracy levels.

```
print("Feature importance:", model.feature_importances_)
```

```
Feature importance: [0.03428109 0.01603486 0.07742074 0.04462701 0.00756603 0.00421043
0.05773247 0.10465483 0.00561231 0.00310465 0.01828184 0.00550746
0.01598658 0.02849289 0.00432991 0.00457692 0.00285285 0.00537969
0.00521666 0.0038365 0.11200226 0.01973492 0.14912901 0.07116392
0.00837159 0.01177922 0.02616743 0.13282975 0.00991387 0.00920232]
```

```
n_features = cancer.data.shape[1]
plt.barh(range(n_features), model.feature_importances_, align='center')
plt.yticks(np.arange(n_features), cancer.feature_names)
plt.xlabel("Feature importance")
plt.ylabel("Feature")
```

Text(0, 0.5, 'Feature')



Note that random forests don't tend to perform well on high dimensional, sparse data, such as text data. Thus, it is recommended to use linear models for such data. Also, they are not easy to explain to non experts, and their random nature makes it difficult to have reproducible results. In such cases you can opt for decision trees. Random forests on large datasets might be time consuming, requiring you to use multiple CPU cores. Alternative ensemble trees such as gradient boosted trees can address this. Where random forests seem to overfit, using alternative trees such as gradient boosted trees would help.

Gradient Boosted Regression Trees

Despite the name, these models can be used for regression and classification. Gradient boosted regression trees don't use any randomization but instead, strong pre-pruning is used. Thus, the trees are shallow (depth one to five), hence using less memory and time.

Unlike random forests, they build trees in a sequential manner with each tree trying to improve on the previous ones. In this model a simple model (known as *weak learners*) is created from a shallow tree and the good predictions from the tree are used to create other trees.

They perform very well hence are very popular, however they need more parameter tuning than random forests to perform that well.

Parameters used are for pre-pruning (`max_depth`) and the number of trees (`n_estimators`) just like random forests, but they also have another important parameter called *learning_rate*, which is the ability of a tree to correct the mistakes of the previous trees. A higher learning rate means each tree can make stronger corrections, allowing for more complex models. Adding more trees to the ensemble, also increases the models' chances to correct mistakes on the training set.

Example

Apply `GradientBoostingClassifier` on the Breast Cancer dataset with 100 trees of maximum depth 1 and a learning rate of 0.1. From the results, it can be seen that without adjusting any parameters the training set accuracy is 100%, a possible sign of overfitting. Limiting the maximum depth of the tree reduces overfitting, while lowering the learning rate increased the generalization performance slightly. We are able to get similar accuracy levels to random forests.


```
#Gradient Boosting on Breast Cancer Dataset
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import GradientBoostingClassifier

cancer = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    cancer.data, cancer.target, stratify=cancer.target, random_state=42)

model = GradientBoostingClassifier(random_state=0)
model.fit(X_train, y_train)

print("Accuracy on training set",model.score(X_train, y_train))
print("Accuracy on testing set",model.score(X_test, y_test))
```

Accuracy on training set 1.0
Accuracy on testing set 0.958041958041958

```
model = GradientBoostingClassifier(random_state=0, max_depth=1)
model.fit(X_train, y_train)

print("Accuracy on training set",model.score(X_train, y_train))
print("Accuracy on testing set",model.score(X_test, y_test))
```

Accuracy on training set 0.9882629107981221
Accuracy on testing set 0.958041958041958

```
model = GradientBoostingClassifier(random_state=0, learning_rate=0.01)
model.fit(X_train, y_train)

print("Accuracy on training set",model.score(X_train, y_train))
print("Accuracy on testing set",model.score(X_test, y_test))
```

Accuracy on training set 0.9882629107981221
Accuracy on testing set 0.9370629370629371

```
model = GradientBoostingClassifier(random_state=0, max_depth=1, learning_rate=0.09)
model.fit(X_train, y_train)

print("Accuracy on training set",model.score(X_train, y_train))
print("Accuracy on testing set",model.score(X_test, y_test))
```

Accuracy on training set 0.9882629107981221
Accuracy on testing set 0.958041958041958

Selecting Parameters

The feature importances of the gradient boosted trees are somewhat similar to those of random forests, though the gradient boosting completely ignores some of the features as shown in the example below. Since both gradient boosting and random forests perform well on similar kinds of data, a common approach is to first try random forests, which work quite robustly with less tuning.

Improved algorithms exist in other packages e.g. the xgboost package would be better for large scale problems because its faster than the scikit-learn implementation of gradient boosted tree.

Gradient boosted decision trees also do not work well on high- dimensional sparse data.

Notice that increasing the number of trees in gradient boost trees may lead to overfitting hence adjusting the learning_rates is better. As you adjust the max_depth in gradient boosted trees, keep it low (less than 5)

```
print("Feature importance:",model.feature_importances_)

Feature importance: [0. 0.00307285 0. 0. 0. 0.00164868 0.
0. 0.16080601 0. 0. 0. 0.
0. 0.00792535 0.0019784 0. 0.00032777 0.
0.00113116 0. 0.18258905 0.02528275 0.24063225 0.06762668
0.00211312 0.0005926 0.00194596 0.29301615 0.00351281 0.00579841]
```

```
n_features = cancer.data.shape[1]
plt.barh(range(n_features), model.feature_importances_, align='center')
plt.yticks(np.arange(n_features), cancer.feature_names)
plt.xlabel("Feature importance")
plt.ylabel("Feature")

Text(0, 0.5, 'Feature')
```

